

GUIA USUARIOS

Objetivo

Crear una aplicación de gestión de usuarios que permita a los clientes registrarse, iniciar sesión y acceder a sus propios datos. Además, los usuarios estarán vinculados con sus respectivas subastas y pujas. Se implementarán medidas de seguridad para asegurar que únicamente los usuarios autenticados puedan crear subastas y realizar pujas.

En definitiva, el objetivo es que el estudiante consolide su comprensión sobre los mecanismos de autenticación y autorización en sistemas backend.

Enunciado

Configuración del entorno:

1. Activamos el env de django que hemos configurado en sesiones anteriores (**djangoServer**) de la siguiente manera:
 - Abre un terminal de **Comand Shell. No vale PowerShell de Windows.**
 - Localízate en esa carpeta:
`cd djangoServer/Scripts`
 - Activa el entorno virtual:
`activate`
2. A continuación, vamos a realizar la instalación de dos dependencias nuevas:
 - El framework **rest_framework_simplejwt** para manejar autenticación basada en JWT en proyectos de Django REST Framework. Proporciona:
 - Generación de tokens (access y refresh).
 - Verificación de tokens JWT.
 - Renovación de tokens.
 - Vistas listas para usar (TokenObtainPairView, TokenRefreshView, etc.).
 - **rest_framework_simplejwt.token_blacklist** es una extensión **opcional** que agrega soporte para una funcionalidad extra: el blacklist de tokens.
 - Permite invalidar tokens de refresh (por ejemplo, cuando un usuario cierra sesión).
 - Usa modelos de Django para guardar los tokens revocados o usados.
 - Para obtener las dos dependencias basta con ejecutar el siguiente comando:
`pip install djangorestframework-simplejwt[token_blacklist]`
3. Damos de alta las dependencias en INSTALLED_APPS de myFirstApiRest/settings.py:

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'rest_framework',  
    'rest_framework_simplejwt',  
    'rest_framework_simplejwt.token_blacklist',  
    # Resto de dependencias  
]
```

4. Indicamos que el framework de APIREST va a utilizar el mecanismo JWT para la autenticación. Para ello modificamos la variable REST_FRAMEWORK para incluir:

```
REST_FRAMEWORK = {  
    # Resto de configuraciones  
    'DEFAULT_AUTHENTICATION_CLASSES': (  
        'rest_framework_simplejwt.authentication.JWTAuthentication',  
    ),  
}
```

5. Posteriormente, crea una nueva variable llamada SIMPLE_JWT para definir las características del token: la duración y el ciclo de vida.

```
from datetime import timedelta  
  
SIMPLE_JWT = {  
    "ACCESS_TOKEN_LIFETIME": timedelta(days=1),  
    "REFRESH_TOKEN_LIFETIME": timedelta(days=7),  
    "ROTATE_REFRESH_TOKENS": True,  
    "BLACKLIST_AFTER_ROTATION": True,  
}
```

Creación de la aplicación de usuarios:

1. Primer paso, crea una nueva aplicación llamado “users” dentro del proyecto myFirstApiRest:
 - `django-admin startapp users`
2. Siempre que creamos una nueva aplicación hay que configurarla en INSTALLED_APPS de myFirstApiRest/settings.py:

```
INSTALLED_APPS = [  
    # Resto de dependencias  
  
    'rest_framework',  
    'rest_framework_simplejwt',  
    'rest_framework_simplejwt.token_blacklist',  
    'drf_spectacular',  
    'auctions',  
    'users',  
]
```

3. Django nos ofrece por defecto un usuario llamado **AbstractUser** que incluye los siguientes campos.

```
class AbstractUser(models.Model):  
    username = models.CharField(max_length=150, unique=True)  
    first_name = models.CharField(max_length=30, blank=True)  
    last_name = models.CharField(max_length=30, blank=True)  
    email = models.EmailField(blank=True)  
    is_staff = models.BooleanField(default=False)  
    is_active = models.BooleanField(default=True)  
    is_superuser = models.BooleanField(default=False)  
    last_login = models.DateTimeField(blank=True, null=True)  
    date_joined = models.DateTimeField(default=timezone.now)
```

Como nosotros necesitamos añadir a esa lista los campos de fecha de nacimiento, municipio y localidad, crearemos un nuevo usuario (llamado **CustomUser**) que herede de **AbstractUser**.

Accede a [users/models.py](#) y añade el siguiente código:

```
from django.db import models  
from django.contrib.auth.models import AbstractUser  
  
class CustomUser(AbstractUser):  
    birth_date = models.DateField()  
    locality = models.CharField(max_length=100, blank=True)  
    municipality = models.CharField(max_length=100, blank=True)
```

Fíjate que estamos poniendo los campos municipio y localidad como **optativos** usando la opción “blank”. Otra forma de hacerlo es con “null”. Veamos cuando usar cada uno:

- En campos **tipo texto** (CharField, TextField), usa **blank=True**. Permite cadenas vacías (“”) pero no NULL.
- En **otros tipos** de campos como fechas, números, FK, usa **null=True**.

4. A continuación, accede a [myFirstApiRest/settings.py](#) y crea una nueva variable con la que indicar al servidor cuál será el modelo de usuario con el que se va a hacer gestionar la autenticación y autorización en el proyecto; el **CustomUser** que hemos definido en el paso anterior.

```
AUTH_USER_MODEL = 'users.CustomUser'
```

5. Dado que hemos realizado un cambio en el modelo, ejecutamos los comandos necesarios para aplicar los cambios a la base de datos. Esto permite aplicar el ORM.

- `py manage.py makemigrations users`

- Este comando se encarga de detectar los cambios que ha habido en los modelos de la aplicación users. **IMPORTANTE:** crea las operaciones, pero no las ejecuta. Eso lo hará con el siguiente comando (**migrate**).

- `py manage.py migrate`

- Este comando es el encargado de aplicar todas las migraciones pendientes de todos los aplicativos del proyecto.

ERROR: Puede que el último comando (**migrate**) nos lance el siguiente error:

```
raise InconsistentMigrationHistory(
    django.db.migrations.exceptions.InconsistentMigrationHistory:
    Migration admin.0001_initial is applied before its dependency
    users.0001_initial on database 'default'.
```

Si os pasa, basta con **borrar** el fichero de base de datos “db.sqlite3” y volver a ejecutar el comando “`py manage.py migrate`”.

Con ello provocamos que vuelvan a crear las tablas de base de datos. Pero se habrá perdido las tuplas con los valores de subastas, pujas y categorías. Usad el fichero “Script bd recarga” adjuntos para volver a añadir tuplas de ejemplo al servidor. Recordad que el comando para acceder a la shell es “`py manage.py shell`”.

6. Después, crea el fichero `users/serializers.py` y configúralo de la siguiente manera:

```
from rest_framework import serializers
from .models import CustomUser

class UserSerializer(serializers.ModelSerializer):
    class Meta:
        model = CustomUser
        fields = ('id', 'username', 'email', 'birth_date', 'municipality',
'locality', 'password')
        extra_kwargs = {
            'password': {'write_only': True},
        }

    def validate_email(self, value):
        user = self.instance # Solo tiene valor cuando se está actualizando
        if CustomUser.objects.filter(email=value).exclude(pk=user.pk if user else
None).exists():
            raise serializers.ValidationError("Email already in used.")
        return value

    def create(self, validated_data):
        return CustomUser.objects.create_user(**validated_data)
```

Como el modelo **CustomerUser** hereda de **AbstractUser**, es necesario definir uno a uno los campos a mostrar en **fields** en lugar de usar el “_all_”.

Observa que se está creando un solo serializador para las 4 operaciones CRUD. Esto obliga a añadir unas series de configuraciones:

- Definir el campo “**password**” del tipo “**write_only**”. Esto implica que la contraseña será un parámetro de entrada a la hora de crear un nuevo usuario o editarlo, pero que nunca se devolverá. Es el caso opuesto a la fecha de creación que NO se podía modificar pero que siempre se devolvía.
- Definir la **validación del email**. El email debe ser un campo único en la base de datos. Es decir, no deben existir varios usuarios con el mismo email.
 - CustomUser.objects.filter(email=value).exists()

No obstante, un usuario puede querer modificar su email. Y hay que validar que si el usuario por error introduce su email actual no lancemos un error. Ahí es cuando tenemos que añadir la exclusión:

 - CustomUser.objects.filter(email=value).exclude(pk=user.pk if user else None).exists():
- Definir la función **create** para validar los datos del usuario que se quiere registrar. Este método implementa por defecto el guardado de contraseña encriptada que proporciona el **AbstractUser**.

7. Posteriormente crearemos las vistas para definir los servicios de CRUD de usuarios. Esto será un poco especial debido a que los usuarios se crean mediante un formulario de registro. Por eso crearemos una vista dedicada.

```
from rest_framework import status, generics
from rest_framework.response import Response
from rest_framework_simplejwt.tokens import RefreshToken
from .models import CustomUser

class UserRegisterView(generics.CreateAPIView):
    queryset = CustomUser.objects.all()
    serializer_class = UserSerializer

    def create(self, request, *args, **kwargs):
        serializer = self.get_serializer(data=request.data)
        if serializer.is_valid():
            user = serializer.save()
            refresh = RefreshToken.for_user(user)
            return Response({
                'user': serializer.data,
                'access': str(refresh.access_token),
                'refresh': str(refresh),
            }, status=status.HTTP_201_CREATED)
        return Response(serializer.errors,
            status=status.HTTP_400_BAD_REQUEST)

class UserListView(generics.ListAPIView):
    serializer_class = UserSerializer
    queryset = CustomUser.objects.all()

class UserRetrieveUpdateDestroyView(generics.RetrieveUpdateDestroyAPIView):
    serializer_class = UserSerializer
    queryset = CustomUser.objects.all()
```

8. Ahora, crea un nuevo fichero llamado “urls.py” en la aplicación *users* para permitir la gestión de usuarios.

```
from django.urls import path
from .views import UserRegisterView, UserListView, UserRetrieveUpdateDestroyView

app_name="users"
urlpatterns = [
    path('register/', UserRegisterView.as_view(), name='user-register'),
    path('', UserListView.as_view(), name='user-list'),
    path('<int:pk>/', UserRetrieveUpdateDestroyView.as_view(), name='user-detail'),
]
```

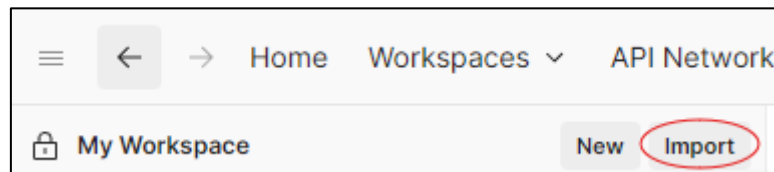
9. Por último, añade la configuración de urls a nivel de proyecto en *myFirstApiRest/urls.py*:

```
from django.contrib import admin
from django.urls import include, path
from drf_spectacular.views import SpectacularAPIView, SpectacularSwaggerView

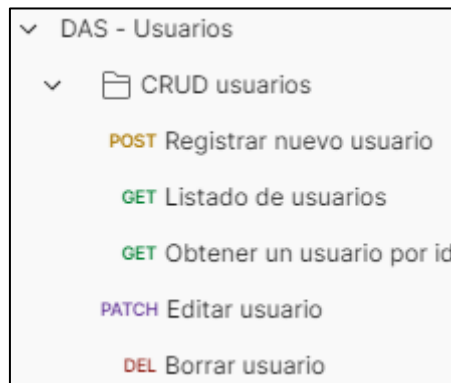
urlpatterns = [
    path("api/auctions/", include("auctions.urls")),
    path("api/users/", include("users.urls")),
    path("admin/", admin.site.urls),
    path('api/schema/', SpectacularAPIView.as_view(), name='schema'),
    path('api/schema/swagger-ui/',
SpectacularSwaggerView.as_view(url_name='schema'), name='swagger-ui'),
]
```

10. Vamos a probar con una colección de Postman el nuevo servicio creado.

- `py manage.py runserver`
- Desde la aplicación de Postman, clicas en “Import” para añadir la colección “DAS – Usuarios”.



- Y prueba a lanzar la subcarpeta CRUD Usuarios.



Autenticación con JWT: ¿Quién soy?

La **autenticación** (authentication) es el proceso de verificar quién eres. En APIs, suele hacerse así:

- Te identificas con tus credenciales (usuario y contraseña).
- El sistema las valida.
- Si son correctas, te da un token (como JWT) para que lo uses en las siguientes peticiones.
 - **Access token**: de corta duración (ej. 5 mins), para acceder a recursos protegidos.
 - **Refresh token**: de larga duración (ej. 1 semana), para obtener nuevos **access tokens**.

Ese token es tu prueba de identidad. Cada vez que haces una petición protegida, el sistema revisa si el token es válido.

¿Por qué es necesario? Un endpoint es una puerta de entrada a tu API. Si no lo proteges, cualquiera (usuario o atacante) podría:

- Obtener datos privados (usuarios, subastas...).
- Modificar o eliminar datos sin autorización.
- Realizar acciones maliciosas (crear spam, sobrecargar el sistema, etc.).

Securizar significa limitar el acceso solo a quienes tienen permiso para usar ese recurso.

Mejores prácticas

- Guardar el JWT solo en memoria (cliente) o cookies seguras.
 - No exponer el **refresh token** innecesariamente.
 - Establecer tiempos de expiración razonables.
 - Se configuró en el paso 5 de Configuración del entorno, en el [settings.py](#).
1. ¡Buenas noticias! Django se encarga de realizar el login por nosotros. Basta con configurar en [myFirstApiRest/urls.py](#) los servicios de token marcados en negrita.

```
from django.contrib import admin
from django.urls import include, path
from drf_spectacular.views import SpectacularAPIView, SpectacularSwaggerView
from rest_framework_simplejwt.views import (TokenObtainPairView, TokenRefreshView)

urlpatterns = [
    path("api/auctions/", include("auctions.urls")),
    path("api/users/", include("users.urls")),
    path('api/token/', TokenObtainPairView.as_view(), name='token_obtain_pair'),
    path('api/token/refresh/', TokenRefreshView.as_view(), name='token_refresh'),
    path("admin/", admin.site.urls),
    path('api/schema/', SpectacularAPIView.as_view(), name='schema'),
    path('api/schema/swagger-ui/', SpectacularSwaggerView.as_view(url_name='schema'),
name='swagger-ui'),
]
```

- **api/token:**
 - Se utiliza para hacer login.
 - Recibe en una petición POST las credenciales del usuario (username y password).
 - Si las credenciales son válidas, responde con dos tokens:
 - access: Token de acceso válido por poco tiempo (por defecto, 5 min).

- refresh: Token de actualización que permite obtener un nuevo token de acceso sin volver a hacer login (por defecto, válido 1 día o más).
- **api/token/refresh:**
 - Se usa para renovar un token de acceso cuando ha expirado.
 - Recibe un refresh token válido.
 - Devuelve un nuevo access token (pero no uno nuevo de refresh).

Ejercicio 1

Usando Postman, prueba a iniciar sesión usando la petición “Login” de la carpeta JWT. Las credenciales a usar son las creadas para registrar un nuevo usuario (de la carpeta CRUD usuarios).

Ejercicio 2

Revisemos el contenido del token JWT obtenido en el ejercicio anterior. Para ello:

- Copie el token de acceso.
- Navegue a la dirección de <https://jwt.is/>
- En la sección “encoded” de la parte superior, pegue el **access_token**.
- En la sección de “Payload data” puede revisar el contenido del token descodificado. ¿De cuántos campos se compone?

¿Qué es APIView?

APIView es una clase base en Django REST Framework que te permite construir vistas personalizadas que manejan solicitudes HTTP (GET, POST, PUT, etc.) de forma explícita, como en Django tradicional, pero con todas las ventajas del framework REST.

Se usa cuando necesitas más control sobre la lógica de la vista que el que te ofrecen los genéricos como CreateAPIView, ListAPIView, etc.

2. Vamos a usar APIView para crear el servicio de log out. Para ello se necesita que se introduzca en el body el token de refresco que se va a invalidar. Al guardar este token en la lista de excluidos (blacklist) se impide que se den de alta nuevos tokens de acceso.

```
from rest_framework import status, generics
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework_simplejwt.tokens import RefreshToken
# Resto de imports

# Resto de vistas
class LogoutView(APIView):

    def post(self, request):
        """Realiza el logout eliminando el RefreshToken (revocar)"""
        try:
            # Obtenemos el RefreshToken del request
            # Se espera que esté en el header Authorization
            refresh_token = request.data.get('refresh', None)
            if not refresh_token:
                return Response({"detail": "No refresh token provided."},
                                status=status.HTTP_400_BAD_REQUEST)

            # Revocar el RefreshToken
            token = RefreshToken(refresh_token)
            token.blacklist()
            return Response({"detail": "Logout successful"},
                            status=status.HTTP_205_RESET_CONTENT)

        except Exception as e:
            return Response({"detail": str(e)}, status=status.HTTP_400_BAD_REQUEST)
```

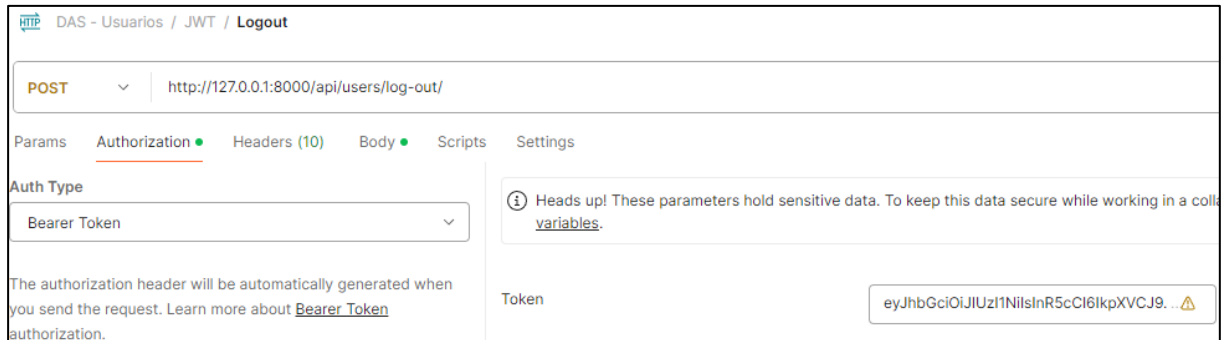
3. Configura este nuevo servicio en users/urls.py:

```
from django.urls import path
from .views import UserRegisterView, UserListView,
UserRetrieveUpdateDestroyView, LogoutView

app_name="users"
urlpatterns = [
    path('register/', UserRegisterView.as_view(), name='user-register'),
    path('', UserListView.as_view(), name='user-list'),
    path('<int:pk>', UserRetrieveUpdateDestroyView.as_view(),
        name='user-detail'),
    path('log-out/', LogoutView.as_view(), name='log-out'),
]
```

Ejercicio 3

Usando Postman, prueba a cerrar sesión usando la petición “Logout” de la carpeta JWT. Las credenciales a usar son las creadas para registrar un nuevo usuario (de la carpeta CRUD usuarios). Recuerda introducir el token de acceso en la pestaña “Authorization” de Postman, con el tipo “Bearer Token”.



Perfil de usuarios

Restringir el acceso a que un usuario solo pueda **ver, modificar o eliminar sus propios datos** es esencial para proteger la privacidad, seguridad y confidencialidad de la información personal.

Permitir el acceso libre a los datos de otros usuarios puede exponer información sensible (como emails, direcciones, historial de actividad, etc.) y viola los principios de protección de datos (como el RGPD en Europa).

Los endpoints creados en el paso anterior permite consultar todas las listas de usuarios existentes, o modificar la información de cualquier cliente. Para delimitar que cada usuario consulte solo su información vamos a crear el servicio “**profile**”.

1. Creamos una nueva vista en users/views.py.

Como mencionamos para la vista de LogOut, vamos a usar APIView. De esta forma definiremos cada uno de los métodos HTTP que vamos a habilitar:

- **Get** para la consulta de la propia información del usuario.
- **Patch** para la actualización parcial de la información del usuario. El cambio de contraseña se gestionará por otro servicio que se implementaremos en pasos posteriores.
- **Delete** para borrar la cuenta del usuario.

Para restringir que la información que se ofrece es la del usuario que realiza la petición, la identificación del usuario no puede ser por un parámetro de entrada (ni *query param* ni en el *body*). Por que se podría sobrescribir fácilmente. La información del usuario actual se obtiene del token de autenticación. Django se encarga de obtener el identificador de usuario del token JWT (como vimos en el ejercicio 2) y cargarte en la *request* la información completa de ese usuario: ***request.user***

```
class UserProfileView(APIView):

    def get(self, request):
        serializer = UserSerializer(request.user)
        return Response(serializer.data)

    def patch(self, request):
        serializer = UserSerializer(request.user, data=request.data, partial=True)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

    def delete(self, request):
        user = request.user
        user.delete()
        return Response(status=status.HTTP_204_NO_CONTENT)
```

2. Añadimos la nueva vista (***UserProfileView***) a ***users/urls.py***:

```
from django.urls import path
from .views import UserRegisterView, UserListView,
UserRetrieveUpdateDestroyView, UserProfileView, LogoutView

app_name="users"
urlpatterns = [
    path('register/', UserRegisterView.as_view(), name='user-register'),
    path('', UserListView.as_view(), name='user-list'),
    path('<int:pk>/', UserRetrieveUpdateDestroyView.as_view(),
name='user-detail'),
    path('profile/', UserProfileView.as_view(), name='user-profile'),
    path('log-out/', LogoutView.as_view(), name='log-out'),
]
```

3. A continuación, probaremos el nuevo servicio usando Postman:

- Accede a la subcarpeta “Perfil” y a la petición “Obtener la información propia del usuario”.
- Ejecútala.
- Dará el error:

```
{
  "detail": "Authentication credentials were not provided."
}
```

- Este error sucede porque no hemos puesto el token de acceso. Por lo que primero deberemos logearnos. Vete a la subcarpeta “JWT”, a la petición de “Login” y obtén el

access_token. Una vez obtenido, cópialo en la pestaña de Authorization. Ejecuta la petición.

- e. El resultado debe ser la información del usuario. Solo un usuario. El del token de autenticación.

Ejercicio 4

Realiza una petición PATCH para modificar algún campo de usuario. ¿Cuál es la diferencia entre PUT y PATCH?

4. Como adelantábamos, el cambio de contraseña se va a realizar mediante un servicio separado. El motivo principal es poder añadir lógica extra al cambio de contraseña. De esta forma pediremos al usuario que proporcione la contraseña antigua y la nueva. Empezamos creando un nuevo serializador en users/serializers.py:

```
class UserSerializer(serializers.ModelSerializer):  
    # Definición del serializador  
  
class ChangePasswordSerializer(serializers.Serializer):  
    old_password = serializers.CharField(required=True)  
    new_password = serializers.CharField(required=True)
```

Fíjate que a diferencia de los otros serializadores que hemos ido declarando, este es un **serializers.Serializer**. Esto se debe a que no tiene un modelo asociado. Si no que se está definiendo en este punto los parámetros de entrada del servicio de cambio de modelo. Dos campos: **old_password** y **new_password**.

5. La lógica del cambio de contraseña se recoge en la vista.

Cuando se realiza una petición POST, la vista recibe los datos del formulario (que deben incluir la contraseña actual y la nueva) y los valida utilizando un serializer. Primero, se verifica que la contraseña actual ingresada sea correcta utilizando el método **check_password**.

Si la contraseña actual es válida, se valida la nueva contraseña usando la función **validate_password** de Django, que comprueba que cumpla con los requisitos de seguridad (longitud mínima, complejidad, etc.). Si la nueva contraseña es válida, se actualiza y guarda en la base de datos mediante **set_password**.

Finalmente, si todo es correcto, devuelve una respuesta indicando que la contraseña fue actualizada con éxito; de lo contrario, devuelve los errores correspondientes. Este enfoque garantiza tanto seguridad como retroalimentación clara al usuario.

```
# Resto de imports
from .serializers import UserSerializer, ChangePasswordSerializer
from rest_framework.exceptions import ValidationError
from django.contrib.auth.password_validation import validate_password

# Resto de vistas
class ChangePasswordView(APIView):

    def post(self, request):
        serializer = ChangePasswordSerializer(data=request.data)
        user = request.user

        if serializer.is_valid():
            if not user.check_password(serializer.validated_data['old_password']):
                return Response({"old_password": "Incorrect current password."},
                                status=status.HTTP_400_BAD_REQUEST)

            try:
                validate_password(serializer.validated_data['new_password'], user)
            except ValidationError as e:
                return Response({"new_password": e.messages},
                                status=status.HTTP_400_BAD_REQUEST)

            user.set_password(serializer.validated_data['new_password'])
            user.save()
            return Response({"detail": "Password updated successfully."})

        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)
```

6. Añadimos la nueva vista (***ChangePasswordView***) a *users/urls.py*:

```
from django.urls import path
from .views import UserRegisterView, UserListView, UserRetrieveUpdateDestroyView,
UserProfileView, ChangePasswordView, LogoutView

app_name="users"
urlpatterns = [
    path('register/', UserRegisterView.as_view(), name='user-register'),
    path('', UserListView.as_view(), name='user-list'),
    path('<int:pk>/', UserRetrieveUpdateDestroyView.as_view(), name='user-detail'),
    path('profile/', UserProfileView.as_view(), name='user-profile'),
    path('change-password/', ChangePasswordView.as_view(), name='change-password'),
    path('log-out/', LogoutView.as_view(), name='log-out'),
]
```

7. A continuación, probaremos el servicio de cambio de contraseña usando Postman:

- Accede a la subcarpeta “Perfil” y a la petición “Cambiar contraseña del usuario”.
- Ejecútala.

Ejercicio 5

Hemos mencionado que el cambio de contraseña realiza una serie de validaciones. Prueba la petición “Cambiar contraseña del usuario” de la subcarpeta “Profile” para provocar excepciones en el cambio de contraseña.

Autorización: ¿Qué puedo hacer?

La **autorización** es el proceso mediante el cual se determina qué acciones puede realizar un usuario dentro de un sistema una vez que ha sido autenticado (es decir, una vez que se ha verificado su identidad). En otras palabras, responde a la pregunta: “¿Qué puede hacer este usuario?”

Para facilitar la asignación de permisos, se suelen definir “roles”. Un rol es una agrupación de acciones permitidas para un usuario.

En nuestra aplicación habrá tres tipos de usuarios:

- Usuario no registrado (no autenticado):** Permite la consulta de subastas, pero no puede pujar. No se ha registrado, por lo que no tiene cuenta que consultar ni modificar.
- Usuario registrado (autenticado):** Permite CRUD de sus propias subastas y pujas. Además, puede consultar su información de usuario y aplicar cambios.
- Administrador (autenticado + permisos elevados):** Es gestor del sitio web y puede modificar subastas y pujas de cualquier usuario. Puede consultar usuarios y modificar la información de otras cuentas.

En resumen, las opciones que pueden hacer:

Funcionalidad	No registrado	Registrado	Administrador
Consultar subastas públicas	☒	☒	☒
Crear subasta	☐	☒	☒
Editar o eliminar sus subastas	☐	☒	☒
Editar o eliminar cualquier subasta	☐	☐	☒
Consultar categorías	☒	☒	☒
Crear/editar/eliminar categorías	☐	☐	☒
Registrarse	☒	☐	☐
Ver/editar/eliminar su perfil	☐	☒	☒
Ver/editar/eliminar otros perfiles	☐	☐	☒
Hacer logout	☐	☒	☒

Existen tres tipos de permisos por defecto proporcionados por django:

Permiso	Descripción	¿Qué usuario puede acceder?
<i>IsAuthenticated</i>	Permite el acceso solo a usuarios autenticados.	Solo usuarios autenticados.
<i>AllowAny</i>	Permite el acceso a cualquier usuario, autenticado o no.	Todos (registrados, no registrados, admin).
<i>IsAdminUser</i>	Permite el acceso solo a usuarios administradores (con is_staff).	Solo administradores (usuarios con privilegios).

1. Configuremos los permisos para la vista de usuarios:

```
from rest_framework.permissions import IsAuthenticated, AllowAny, IsAdminUser
# Resto de imports

class UserRegisterView(generics.CreateAPIView):
    permission_classes = [AllowAny]
    # Resto de la vista

class UserListView(generics.ListAPIView):
    permission_classes = [IsAdminUser]
    # Resto de la vista

class UserRetrieveUpdateDestroyView(generics.RetrieveUpdateDestroyAPIView):
    permission_classes = [IsAdminUser]
    # Resto de la vista

class UserProfileView(APIView):
    permission_classes = [IsAuthenticated]
    # Resto de la vista

class ChangePasswordView(APIView):
    permission_classes = [IsAuthenticated]
    # Resto de la vista

class LogoutView(APIView):
    permission_classes = [IsAuthenticated]
    # Resto de la vista
```

Ejercicio 6

Accede a la shell de django y consulta una instancia de CustomUser utilizando los comandos proporcionados en "Script bd usuarios.txt". ¿Qué campo se utiliza para saber si un usuario es administrador o no?

Ejercicio 7

Ahora que hemos establecido los diferentes niveles de autorización, intenta acceder con usuario NO administrador al listado de usuarios. ¿Cuál es el error obtenido?

Ejercicio 8

Cambia el tipo de usuario de “restándar” a administrador mediante los siguientes comandos y repite el ejercicio 7.

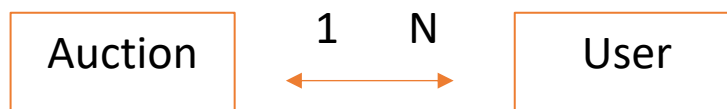
```
from django.contrib.auth.models import User

user = User.objects.get(id=1)

# Esto le da permisos para acceder al panel de administración.
user.is_staff = True

user.save()
```

2. Se continúa con la configuración de permisos para las subastas. Primero de todo es añadir la relación entre usuarios y subastas.



Cada usuario registrado puede crear tantas subastas como quiera. Y una subasta pertenece de forma unívoca a un usuario. Esta relación es del tipo 1 – N que ya hemos trabajado con categorías y pujas. Por tanto, creamos el campo “**auctioneer**” en subastas para reflejar la relación con el usuario, en el fichero users/models.py:

```
from users.models import CustomUser
# Resto de imports

# Resto de modelos
class Auction(models.Model):
    # Resto de campos del modelo

    auctioneer = models.ForeignKey(CustomUser, related_name='auctions',
on_delete=models.CASCADE)

    class Meta:
        ordering=('id',)

    def __str__(self):
        return self.title
```

3. Como siempre que hemos realizado un cambio en el modelo, ejecutamos los comandos necesarios para aplicar los cambios a la base de datos. Esto permite aplicar el ORM.

- `py manage.py makemigrations users`
- `py manage.py migrate`

WARNING: En este cambio hemos creado una nueva columna en una tabla de base de datos que ya tenía campos insertados (las 10 subastas). Como la relación 1-N es obligatoria, django no sabe con qué valor rellenar esta columna para las subastas que ya están declaradas en la base de datos y nos ofrece estas dos opciones:

It is impossible to add a non-nullable field 'auctioneer' to auction without specifying a default. This is because the database needs something to populate existing rows.

Please select a fix:

- 1) Provide a one-off default now (will be set on all existing rows with a null value for this column)
- 2) Quit and manually define a default value in models.py.

Selecciona la **opción 1**, que indica ponerles a todas las subastas un valor por defecto para el subastador. Posteriormente, indicad el identificador de un usuario que tengáis en base de datos; por ejemplo, el 1.

4. Damos de alta la nueva vista que muestra las subastas de un usuario en [auctions/views.py](#):

```
from rest_framework.views import APIView
from rest_framework.permissions import IsAuthenticated
# Resto de imports

# Resto de vistas
class UserAuctionListView(APIView):
    permission_classes = [IsAuthenticated]

    def get(self, request, *args, **kwargs):
        # Obtener las subastas del usuario autenticado
        user_auctions = Auction.objects.filter(auctioneer=request.user)
        serializer = AuctionListCreateSerializer(user_auctions, many=True)
        return Response(serializer.data)
```

Se filtra por las subastas cuyo creador (o subastador) es el usuario que realiza la petición.

5. Y damos de alta la vista en [auctions/urls.py](#):

```
from django.urls import path
from .views import CategoryListCreate, CategoryRetrieveUpdateDestroy,
AuctionListCreate, AuctionRetrieveUpdateDestroy, BidListCreateView, BidDetailView,
UserAuctionListView

app_name="auctions"
urlpatterns = [
    path('categories/', CategoryListCreate.as_view(), name='category-list-create'),
    path('categories/<int:pk>/', CategoryRetrieveUpdateDestroy.as_view(),
name='category-detail'),
    path('', AuctionListCreate.as_view(), name='auction-list-create'),
    path('<int:pk>/', AuctionRetrieveUpdateDestroy.as_view(), name='auction-detail'),
    path('<int:id_auction>/bid/', BidListCreateView.as_view(), name='bid-list-
create'),
    path('<int:id_auction>/bid/<int:pk>/', BidDetailView.as_view(), name='bid-
detail'),
    path('users/', UserAuctionListView.as_view(), name='action-from-users'),
]
```

6. Usa la aplicación Postman para validar el nuevo servicio. En la subcarpeta “Autorización” puedes encontrar la petición “Obtener subastas del usuario”. Recuerda poner el token de autorización.
Como hemos definido que todas las subastas son del mismo usuario, saldrán todas las subastas.

Ejercicio 9

Crea un nuevo usuario. A su vez, loggeate y crea una nueva subasta. Comprueba que si llamas a <http://127.0.0.1:8000/api/auctions/users> solo se muestra una subasta.

7. Por último, nos faltaría asignar los permisos restantes a los servicios de la aplicación de subastas.
No obstante, los permisos que hemos vistos (*IsAuthenticated*, *AllowAny*, *IsAdminUser*) no encajan adecuadamente con el requisito: que solo se pueda modificar datos si eres el creador o tienes rol administrador. Esto tiene sencilla solución: crear un nuevo tipo de permiso.
Para ello, crea en la aplicación de subastas un nuevo fichero llamado `permissions.py` ([auctions/permissions.py](#)):

```
from rest_framework.permissions import BasePermission, SAFE_METHODS

class IsOwnerOrAdmin(BasePermission):
    """
    Permite editar/eliminar una subasta solo si el usuario es el propietario
    o es administrador. Cualquiera puede consultar (GET).
    """

    def has_object_permission(self, request, view, obj):
        # Permitir acceso de lectura a cualquier usuario (GET, HEAD, OPTIONS)
        if request.method in SAFE_METHODS:
            return True

        # Permitir si el usuario es el creador o es administrador
        return obj.auctioneer == request.user or request.user.is_staff
```

8. Modifica la vista para añadir el nuevo permiso:

```
from .permissions import IsOwnerOrAdmin
# Resto de imports

# Resto de vistas
class AuctionRetrieveUpdateDestroy(generics.RetrieveUpdateDestroyAPIView):
    permission_classes = [IsOwnerOrAdmin]
```

Ejercicio 10

Añade los permisos restantes a las vistas de subastas; incluyendo categorías y pujas. Una vez configurado, valida mediante llamadas al servidor desde Postman que cumple lo esperado.

Ejercicio 11

Modela la relación entre usuarios y pujas. Sabiendo que un usuario puede pujar todas las veces que quiera sobre una subasta (teniendo que ser la nueva puja mayor que las que ya existan de ese usuario o de cualquier otro) y que una subasta pertenece a un usuario. Deberá incluir las restricciones de autorización definidas a continuación:

Funcionalidad	No registrado	Registrado	Administrador
Consultar pujas públicas	☒	☒	☒
Crear puja	☒	☒	☒
Editar o eliminar sus pujas	☒	☒	☒
Editar o eliminar cualquier puja	☒	☒	☒